

Alon Anthony
3453 Wheeling Drive, 95051, Santa Clara CA , United States,
webmaster@synpat.com,
(408) 480 31 86,
www.ip-consult.net,

Report of Evidence of Use

The views presented here are of the author only and do not necessarily reflect the views of others. The author is an independent contractor that acts on behalf of neither the patent owner nor any other third party, including ClaimCharts.info. We would like to thank ClaimCharts.info for introducing to us the opportunity to prepare this report. For comments and questions, please feel free to contact us directly, or via my profile at ClaimCharts.info.

Patent Number: USUS8909824

Claim Number: (1

1. A receiver for processing data fragments from a transmitter using a single timer for one connection between the receiver and the transmitter, the receiver comprising: a first module for processing the fragments according to a processing index in each of the fragments, wherein the each of the fragments has one or more data blocks, the processing index in the each of the fragments is to facilitate the fragments to be processed sequentially, the each of the fragments is processed till one or more of predefined conditions is met, the predefined conditions include: the timer assigned to the each of the fragments is timed out or an acknowledgement of at least one of the data blocks in the each of the fragments is received; and a second module designed to start the timer assigned to the each of the fragments, wherein the timer has a timeout value depending on a life of the each of the fragments and time parameters thereof , wherein the timer is running until processing of the each of the fragments is terminated, the timer is then reassigned to a next one of the fragments before the next one of the fragments is processed.

Element 1:

1. A receiver for processing data fragments from a transmitter using a single timer for one connection between the receiver and the transmitter, the receiver comprising:

CLOCKLESS VIRTUAL GPIO

Application number: 20150149672
Abstract: A virtual GPIO architecture for an integrated circuit is provided that both serializes virtual GPIO signals and deserializes virtual GPIO signals without the need for an external clock.
Type: Application
Filed: November 13, 2014

Issued: May 28, 2015
Inventors: Lalan Jee Mishra, Richard Dominic Wietfeldt, Mohit Kishore Prasad

SYSTEMS AND METHODS FOR BIASING A BUS

Application number: 20150149671
Abstract: A bi-directional differential bus interface that includes a differential transmitter having a non-inverting terminal and an inverting terminal, a differential receiver having a non-inverting terminal and an inverting terminal, and a biasing circuit that is electrically coupled to the non-inverting terminal of the differential transmitter and the inverting terminal of the differential transmitter. The biasing circuit is configured to generate a voltage between the non-inverting terminal of the differential transmitter and the inverting terminal of the differential transmitter that is approximately 200 mV or more in response to assertion of a control signal received on a control input of the biasing circuit.
Type: Application
Filed: May 21, 2012
Issued: May 28, 2015
Assignee: SCHNEIDER ELECTRIC IT CORPORATION
Inventors: Daniel J. Rohr, Richard Earl Jenkins

METHOD AND APPARATUS FOR USE IN A DATA PROCESSING SYSTEM

Application number: 20150143005
Abstract: A method is described, for use in a data processing system, the system having a node and a communication link, wherein the communication link is coupled to the node. The method can comprise obtaining first digital signal information associated with a first signal, transmitting the first signal from the node to the communication link, receiving a second signal from the communication link at the node, and analysing the second signal to obtain second digital signal information. The method can further include combining first digital signal information with second digital signal information and flagging a combination outside a predetermined condition space. Further, an apparatus, for use in the data processing system is described that can be operative to perform the method. A data processing system is also described.
Type: Application
Filed: November 18, 2013
Issued: May 21, 2015
Assignee: Infineon Technologies AG
Inventor: Achim VOWE

MULTI-PROTOCOL COMBINED RECEIVER

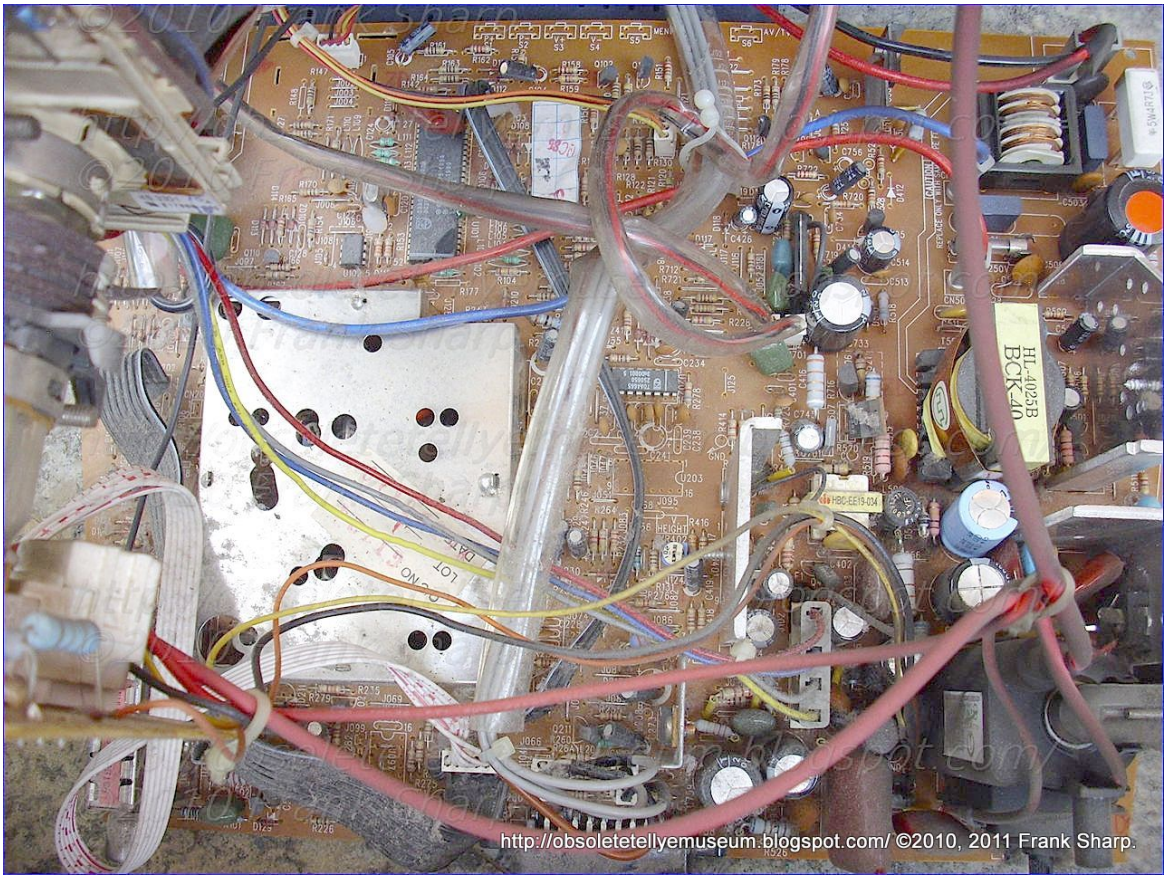
Application number: 20150143006
Abstract: A receiver circuit configured to operate in a DisplayPort (DP) mode and a High-Definition Multimedia Interface (HDMI) mode. The receiver circuit includes: termination circuitry configured to receive a DP signal in the DP mode and an HDMI signal in the HDMI mode; and voltage common-mode (VCM) level shifter circuitry configured to operate as a pass-through for the DP signal in the DP mode and generate a converted HDMI signal from the HDMI signal in the HDMI mode.
Type: Application
Filed: November 19, 2013
Issued: May 21, 2015
Assignee: Synaptics Incorporated
Inventor: Yonggang Chen

METHOD FOR SYNCHRONIZING DISPLAY ELEMENTS IN A CONTROL SYSTEM FOR AUTOMATED CONTROL OF A TECHNICAL INSTALLATION

Application number: 20150134865
Abstract: A method for synchronizing display elements of a plurality of subscribers of a distributed, bus-based control system is proposed. The subscribers comprise a message transmitter and a plurality of message receivers. The message receivers each comprise a respective display element, which can be toggled between a switched-on state and a switched-off state, and a control unit for producing a control signal for toggling the display element from one state to the other. The message transmitter sends a respective current bus message via the bus in defined recurring bus message time intervals. The respective bus message comprises a bus message counter that has a respective value from a defined value range. The counter value is changed in a predefined manner each time when a current bus message is sent. The message receivers receive the respective bus messages via the bus.
Type: Application
Filed: January 19, 2015
Issued: May 14, 2015
Inventors: Thorsten GODAU, Christian ADAMS, Reinhard SPERRER, Dietmar SEIZINGER

SUPERSPEED INTER-CHIP INTERFACE

Application number: 20150134866
Abstract: An interface for low power, high bandwidth communications between units in a device in provided herein. The interface comprises a USB 3.0 system interface and a SuperSpeed inter-chip (SSIC) protocol adaptor configured to facilitate communications between the USB 3.0 system interface and an M-PHY interface.
Type: Application
Filed: January 20, 2015
Issued: May 14, 2015
Inventors: Sridharan Ranganathan, David J. Harriman, Anoop Mukker, Satheesh Chellappan, Karthi R. Vadivelu, Shalini Sharma, Zeeshan Sarwar



Element 2:

a first module for processing the fragments according to a processing index in each of the fragments,

Accumulating evidence shows that brain-derived neurotrophic factor (BDNF) and its receptor tropomyosin-related kinase B (TrkB) significantly decrease early in Alzheimer's disease (AD). However, it remains unclear whether BDNF/TrkB reductions may be mechanistically involved in the pathogenesis of AD. To address this question, we generated 5XFAD transgenic mice with heterozygous TrkB knockout (TrkB$^{+/-}$·5XFAD), and tested the effects of TrkB reduction on AD-like features in this mouse model during an incipient stage that shows only modest amyloid-β (Aβ) pathology and retains normal mnemonic function. TrkB$^{+/-}$ reduction exacerbated memory declines in 5XFAD mice at 4–5 months of age as assessed by the hippocampus-dependent spontaneous alternation Y-maze task, while the memory performance was not affected in TrkB$^{+/-}$ mice. Meanwhile, TrkB$^{+/-}$·5XFAD mice were normal in nest building, a widely used measure for social behavior, suggesting the memory-specific aggravation of AD-associated behavioral impairments. We found no difference between TrkB$^{+/-}$·5XFAD and 5XFAD control mice in cerebral plaque loads, Aβ concentrations including total Aβ42 and soluble oligomers and β-amyloidogenic processing of amyloid precursor protein. Interestingly, reductions in hippocampal expression of AMPA/NMDA glutamate receptor subunits as well as impaired signaling pathways downstream to TrkB such as CREB (cAMP response element-binding protein) and Akt/GSK-3β (glycogen synthase kinase-3β) were observed in TrkB$^{+/-}$·5XFAD mice but not in 5XFAD mice. Among these signaling aberrations, only Akt/GSK-3β dysfunction occurred in TrkB$^{+/-}$ mice, while others were synergistic consequences between TrkB reduction and subthreshold levels of Aβ in TrkB$^{+/-}$·5XFAD mice. Collectively, our results indicate that reduced TrkB does not affect β-amyloidosis but exacerbates the manifestation of hippocampal mnemonic and signaling dysfunctions in early AD.

Brain-derived neurotrophic factor (BDNF) and its receptor tropomyosin-related kinase B (TrkB) decline in brains of mild cognitive impairment or early Alzheimer's disease (AD).[1, 2] As BDNF/TrkB signaling is responsible for neuronal growth, protection and plasticity,[3, 4] its dysfunction as a consequence of amyloid-β (Aβ) accumulation in brain is hypothesized to underlie synaptic and cognitive impairments associated with AD.[5, 6, 7] In accordance with this hypothesis, central BDNF administration or gene delivery in rodent and primate models of AD produces beneficial effects against Aβ-induced neurotoxicity, synapse loss, aberrant gene expression, deficient cell signaling and impairments in learning and memory.[8, 9] Furthermore, recent work from our laboratory and others demonstrates that systemic administration of 7,8-dihydroxyflavone (7,8-DHF), a small-molecule TrkB agonist, in amyloid precursor protein (APP) transgenic mice improves their synaptic loss and dysfunction, and memory deficits in a battery of hippocampus-dependent paradigms,[10, 11, 12, 13]representing a feasible therapeutic approach that is more efficacious and far less invasive than viral or recombinant BDNF delivery.

Meanwhile, there is also experimental evidence suggesting that BDNF/TrkB signaling may be an important regulator of amyloidogenic processing. For example, Aβ production in cultured cells is reduced by BDNF treatment,[14] while it is facilitated by BDNF deprivation.[15] Chronic activation of TrkB receptors with 7,8-DHF is reported to exert beneficial effects in APP mice, at least in part, by alleviating cerebral Aβ accumulation.[10, 11] The pathogenic role of deficient BDNF/TrkB signaling is also supported by the observations that environmental risks for sporadic AD (for example, aging, stress, high-fat diet and so on) impair BDNF/TrkB pathways,[16, 17, 18] whereas factors that may reduce AD risks (for example, exercise, environmental enrichment, social interaction, dietary restriction and so on) elevate BDNF/TrkB levels.[13, 19, 20, 21, 22, 23] Moreover, accumulating data suggest that BDNF polymorphisms (for example, Val66Met, Cys270Thr and so on) may be associated with an increased risk of developing AD.[5, 24, 25] In view of these findings, the present study was undertaken to rigorously determine the mechanisms by which deficient BDNF/TrkB signaling may be

responsible for inducing or accelerating AD and related cognitive impairments. Specifically, after crossing TrkB$^{+/-}$ knockout and 5XFAD transgenic mice, we examined the effects of TrkB haploinsufficiency (that is, 50% reduction) on the development of AD-like phenotypes, including cognitive impairments, Aβ accumulation and aberrant hippocampal signaling pathways, in young 5XFAD mice that show only modest plaque pathology and still retain normal learning and memory function.

Animals

We used 5XFAD mice (Tg6799 line) that co-overexpress familial AD (FAD) mutant forms of human APP (Swedish mutation: K670N and M671L; Florida mutation: I716V and London mutation: V717I) and presenilin 1 (PS1; M146L and L286V mutations) transgenes under transcriptional control of the neuron-specific Thy-1 promoter.[26, 27, 28] 5XFAD line was maintained by crossing hemizygous transgenic mice with C57Bl/6 breeders (Taconic, Hudson, NY, USA). To compare changes in nesting behavior in different age groups of 5XFAD mice, 5XFAD hemizygotes were used for the experiment with non-transgenic wild-type littermate mice served as controls. For the study of the TrkB agonist 7,8-DHF, 5XFAD transgenic and wild-type mice at 12 months of age received repeated intraperitoneal administration of 5 mg kg^{-1} 7,8-DHF or 17% dimethyl sulfoxide vehicle once daily for 10 consecutive days, as described previously.[10]

Although TrkB homozygous knockout mice die by 2–3 weeks of age, heterozygous mice survive normally into adulthood.[29, 30] TrkB$^{+/-}$ knockout mice were used as an animal model to recapitulate partial reduction of TrkB expression observed in mild cognitive impairment or incipient AD brains.[1] For our study, hemizygous 5XFAD transgenic mice were crossbred to TrkB$^{+/-}$ mice (B6/129S2 background)[31] (stock number: 002544, Jackson Laboratory, Bar Harbor, ME, USA), yielding animals with four different genotypes (wild type, TrkB$^{+/-}$, 5XFAD$^{+/-}$ and TrkB$^{+/-}$·5XFAD$^{+/-}$). Genotyping was performed by PCR analysis of tail DNA and all experiments were done blind with respect to the genotype of mice. Procedures were conducted in accordance with the National Institutes of Health *Guide for the Care and Use of Laboratory Animals* and approved by the Nathan Kline Institute Animal Care and Use Committee.

Behavioral tests

The mice were tested for the spontaneous alternation Y-maze and nest-building tasks. After behavioral testing, some mice were killed for immunoblotting and enzyme-linked immunosorbent assay (ELISA) experiments, and others were perfused for immunohistochemistry.

Spontaneous alternation Y-maze test

Spontaneous alternation performance was tested using a symmetrical Y-maze, as described previously.[32, 33] Each mouse was placed in the center of the Y-maze and was allowed to explore freely through the maze during an 8-min session. The sequence and total number of arms entered were recorded. Arm entry was considered to be complete when the hind paws of the mouse had been completely placed in the arm. Percentage alternation is the number of triads containing entries into all three arms divided by the maximum possible alternations (the total number of arms entered minus 2) × 100. As the re-entry into the same arm was not counted for analysis, the chance performance level in this task was 50% in the choice between the arm mice visited more recently (non-alternation) and the other arm they visited less recently (alternation).

Nest building

Nest-building behavior of mice was tested as described previously.[34, 35] Mice were individually housed for at least 24 h in clean plastic cages with ~1 cm of corn cob bedding lining the floor and identification cards coded to render the experimenter blind to the sex and genotype of each mouse. Two hours before the onset of the dark phase of the lighting cycle, each cage was supplied a commercially available compressed cotton square (Nestlet, 5 × 5 cm, Ancare,

Bellmore, NY, USA). The next morning (~16 h later), cages were inspected for nest construction. Pictures were taken before evaluation for documentation. Nest construction was scored using the established system of Deacon with a 5-point scale.[35] In brief, the scores were as follows: (1) Nestlet not noticeably touched (>90% intact); (2) Nestlet partially torn (50–90% remaining intact); (3) Nestlet mostly shredded but often no identifiable nest site; (4) an identifiable but flat nest; (5) a (near) perfect nest with clear crater (please see the protocol by Deacon[35] for more detailed scoring standard).

Immunoblot analysis

Hemibrain samples (excluding the cerebellum) or hippocampal samples were taken from the mice under deep isoflurane anesthesia and were snap-frozen for biochemical assays. For western blot analysis, each sample was homogenized in eightfold volumes of cold homogenization medium containing 70 mM sucrose, 210 mM mannitol, 2 mM HEPES, 0.1 mM EDTA and protease phosphatase inhibitor cocktail, and centrifuged at 10 000 *g* for 10 min to remove any insoluble material. Protein concentrations were determined by a BCA protein assay kit (Pierce, Rockford, IL, USA), and 10–50 μg of protein was run on NuPAGE 4–12% or 10%Bis-Tris gels (Invitrogen, Carlsbad, CA, USA) and transferred to nitrocellulose membrane. After blocking, membranes were probed with the following primary antibodies: anti-TrkB (1:1000, ab51190, Abcam, Cambridge, MA, USA), anti-phospho-TrkB (Y705, 1:500, ab52191, Abcam), anti-BACE1 (1:1000, B0681, Sigma-Aldrich, St Louis, MO, USA), an antibody that recognizes C-terminal epitope in APP (1:1000, C1/6.1, kindly provided by Dr Paul Mathews, Nathan Kline Institute) to detect full-length APP/C-terminal fragments, an antibody specific for the β-secretase-cleaved soluble ectodomain of APP (sAPPβ) (1:1000, SIG-39138, Covance, Princeton, NJ, USA), anti-phospho-CREB (cAMP response element-binding protein; Ser133, 1:1000, no. 9198, Cell Signaling Technology, Danvers, MA, USA), anti-CREB (1:2000, no. 9197, Cell Signaling Technology), anti-phospho-Akt (Ser473, 1:1000, no. 9271, Cell Signaling Technology), anti-Akt (1:1000, no. 4685, Cell Signaling Technology), anti-phospho-GSK-3β (glycogen synthase kinase-3β; Ser9, 1:1000, no. 9331, Cell Signaling Technology), anti-GSK-3β (1:2000, sc-9166, Santa Cruz Biotechnology, Dallas, TX, USA), anti-GluA1 (1:1000, no. 13185, Cell Signaling Technology), anti-GluA2 (1:1000, no. 5306, Cell Signaling Technology), anti-NR1 (1:1000, no. 5704, Cell Signaling Technology), anti-NR2A (1:2000, no. 4205, Cell Signaling Technology), anti-NR2B (1:2000, no. 4212, Cell Signaling Technology) and anti-β-actin (1:15 000, AC-15, Sigma-Aldrich). The membranes were then incubated with horseradish peroxidase-conjugated secondary IgG. Immunoblot signals were visualized by an ECL chemiluminescence substrate reagent kit (Pierce) and quantified by densitometric scanning and image analysis using Quantity One software (Bio-Rad Laboratories, Hercules, CA, USA).

ELISAs of soluble Aβ oligomers and total Aβ42

To measure the concentrations of soluble Aβ oligomers, each hemibrain sample was homogenized in eightfold volumes of homogenization medium, as described above. To quantitate total levels of Aβ42, the other hemibrain was extracted in eightfold volumes of cold 5 M guanidine HCl plus 50 mM Tris HCl (pH 8.0) buffer and centrifuged at 20 000 *g* for 1 h at 4 °C to remove insoluble material. Final guanidine HCl concentrations were <0.1 M. Protein concentrations were determined by a BCA protein assay kit (Pierce). Supernatant fractions were analyzed by well-established human Aβ ELISA kits specific to oligomeric forms of Aβ (27725, IBL America, Minneapolis, MN, USA) and Aβ42 (KHB3441, Invitrogen) according to the protocols of the manufacturers. Optical densities at 450 nm of each well were read on a VersaMax tunable microplate reader (Molecular Devices, Sunnyvale, CA, USA), and sample Aβ oligomer and Aβ42 concentrations were determined by comparison with the respective standard curves. Aβ oligomer and Aβ42 concentration values were normalized to total brain protein concentrations and expressed in picograms and nanograms per milligram of total protein, respectively.

Aβ immunohistochemistry

Mice were transcardially perfused with 0.1 M phosphate-buffered saline (PBS; pH7.4), followed by 4% paraformaldehyde in PBS under deep isoflurane anesthesia. Brains were postfixed for 24 h in 4% paraformaldehyde in PBS at 4 °C and transferred to PBS. The brain was sectioned coronally at 30 µm using a vibratome (VT1200, Leica Microsystems, Wetzlar, Germany), and successive sections were stored in PBS containing 0.05% sodium azide at 4 °C. Two sections per mouse taken at levels between –1.7 and –1.9 mm to bregma according to the mouse brain atlas of Franklin and Paxinos[36] were stained by the avidin–biotin peroxidase complex method as described previously.[33, 37, 38] In brief, the sections were incubated overnight at 4 °C with mouse monoclonal anti-Aβ1–16 (6E10) antibody (1:200, SIG-39347, Covance). The avidin–biotin peroxidase complex kit (PK-2200, Vector Laboratories, Burlingame, CA, USA) was utilized with 3,3′-diaminobenzidine tetrahydrochloride as a chromogen to visualize the reaction product. The sections were then mounted on charged slides, dehydrated in a series of alcohol, cleared in xylene and covered with a coverslip. Light microscopy was conducted on an Axioskop 2 microscope equipped with an AxioCaM HRc digital camera (Zeiss, Oberkochen, Germany) for capturing images. Semi-quantitative analysis was performed using AxioVision imaging software with the AutoMeasure module (Zeiss). The threshold optical density that discriminated staining from background was determined and held constant for all quantifications. Identified objects were individually inspected by the same investigator to confirm the object as a plaque or not in a blinded manner. Percentage area occupied by Aβ deposits in the hippocampus and cortex was assessed bilaterally to compare plaque burden between 5XFAD control and $TrkB^{+/-}$·5XFAD mice.

Statistical analysis

Data were analyzed by a one-way or two-way analysis of variance, and *post hoc*Bonferroni comparisons were performed to determine the significance of differences between the groups when appropriate. Data were presented as mean±s.e.m. and the level of significance was set for $P<0.05$.

Element 3:

wherein the each of the fragments has one or more data blocks,



The visitors were dressed as – among other characters – the Mad Hatter and the March Hare

from *Alice's Adventures in Wonderland*, whose 150th anniversary is being celebrated across Britain this year. Oxford, where Lewis Carroll spent much of his life, is the main focus, but Yorkshire has a better claim on the author than many places that make a bigger fuss of him.

Ripon, Whitby, Richmond and Beverley all have Carroll links… but Croft, one of his boyhood homes, can outdo them all, staking a claim for him as a Northern writer. Cheshire-born Carroll – aka Charles Lutwidge Dodgson – lived in Croft's sizeable rectory with his parents and 10 brothers and sisters from 1843, when he was 11 years old, to 1851, when he left for Oxford.

Across the rectory garden is the church, a long, low building featuring a carving which – possibly, conceivably, credibly – may have inspired the Cheshire Cat. Whether this is true or not, Carroll reputedly wrote the first verse of his *Jabberwocky* while living here.

Llandudno in North Wales – which Carroll is never known to have visited – has launched a digital Alice trail on the strength of Alice Liddell (the book's inspiration) holidaying there. So Croft could easily justify a fully-fledged Wonderland World theme park (well, sort of).



It could convert the petanque court next to its village hall into a croquet ground, complete with wooden flamingos for mallets and plastic hedgehogs for balls. It could be the starting point of a Whimsical Wander in Wonderland trail round Yorkshire. But Yorkshire doesn't really do whimsy, doesn't really venture too far down the rabbit hole.

"People from outside the village have wondered why we don't make more of Lewis Carroll," says Coates, who has organised a summer of Alice-themed events in Croft to celebrate the anniversary. "But some here are a little wary; they say: 'We don't want to be invaded.'"

So the only obvious evidence of his connection to the village is kept low-key – a street called Lewis Close and another called Carroll Place. The coaches of Carroll pilgrims roll in from time to time, over a bridge straddling the Tees, the historic boundary between Yorkshire and County Durham (or Darlington Unitary Authority, as the looking-glass world of local government has numbingly renamed this part of the county).

Most tourists aren't British; they come from Japan, Germany, Australia, Russia, the US. "They're absolutely full of it in the way we're not," says Sue. "Even in South Korea there's an Alice in Wonderland teashop."



Some Carroll fans knock boldly on the front door of the rectory, where in his Oxford days he spent happy vacations until his father, the rector, died in 1868 and the family moved out. Carroll wrote a poem – not a very good one – about it:

The Rectory of Croft,

The sun shines bright upon it,

The breezes whisper soft.

He staged magic lantern and conjuring shows there, wrote family magazines and photographed his brothers and sisters, along with visitors, including Tennyson's niece. Copies of some of the photographs hang in the hall of the imposing 18th century rectory: girls with ringlets, ladies in bonnets and crinolines, men with frock coats and fob watches. Some are elegantly posed around a sundial; it's still here, as is an acacia tree that figures in other pictures.

Peter and Jane Atkinson have lived in the rectory for 35 years (they're about to move, ironically to Oxford). When they bought it, it had been converted into three flats, but they converted it back into a family home.

"We didn't buy the house because of its Lewis Carroll association, but it's been a wonderful added extra," says Peter. "People are thrilled to see it, and to 'feel' the house."

Carroll's father, who is buried in the churchyard, thought the rectory had "the elements of abundant comfort". It was big enough to have a wine cellar, two separate libraries, a needlework room and substantial servants' quarters behind a green baize door.

Peter Atkinson leads a tour round its maze of corridors, up back staircases, past a window with the initials "CLD" scratched on it. Charles Lutwidge Dodgson? "If it is," he says, with a smile.

We see Carroll's bedroom with its corner fireplace, and the nursery, where Peter lifts a floorboard to reveal the hiding place where, in 1950, a cache of Victorian relics was found: the lid of a doll's teapot, a letter from a child's alphabet, a piece of clay pipe, a thimble, a small white glove, a child's shoe, a fragment of wood with Carroll's handwriting apparently on it.

And there's a skylight on which workmen scratched their names in the 1830s. From inside, you read them back-to-front. Did this inspire Carroll's interest in mirror-writing? Who knows?

Who also knows how much the sandy-stoned medieval church, with its roses round the porch,

influenced him? It's a fine, eccentric place, dominated by a huge 17th century curtained pew supported by columns and likened over the years to a house on stilts or the royal box in a theatre.

There's a plaque to Carroll, complete with White Rabbit, and an enormous tomb with growling lions guarding its corners; a list of rectors dating back to the splendidly named 13th century Cynicus; and memories of Byron, who spent his honeymoon near Croft after marrying a local girl.

The most intriguing feature, though, is the medieval sedilia, the group of clergy seats on the chancel wall. It's decorated with a surreal jumble of figures and faces, swirling round in the stone, with angels and dragons and trees and pigs snuffling for acorns. And, fantasy piling on fantasy, there's a grinning animal that's a dead ringer for the Cheshire Cat.

You could build a global tourism industry on that alone. And isn't there a precedent for a Yorkshire village church with attached rectory/parsonage and a few literary associations?

Croft's Alice-in-Wonderland summer includes a Mad Hatter's Pimm's & Scones Party (June 20), a new adaptation of Alice's Adventures in Wonderland (June 27), a Carroll-themed Scarecrow Weekend (July 11 & 12), a magic lantern show (September 15) and a Queen of Hearts Ball (October 17). All events free; contact Sue Coates (01325 721014) for details. The informative Lewis Carroll, Child of the North by Anne Clark Amor, is on sale at the church, priced at £5.

Element 4:

the processing index in the each of the fragments is to facilitate the fragments to be processed sequentially,

JSR-166 helped bring Doug Lea's concurrency framework into Java Standard Edition 5. This work carried forward into Java SE 6 as *jsr166x*, which added a number of features to the Java collections framework for concurrency.

Although the contents of Java SE 7 are still in flux, early candidates for inclusion are are already taking shape in *jsr166y*: a fork/join framework and a transfer queue. InfoQ spoke with Doug Lea about these features and concurrency in Java SE 7.

The fork/join framework is a "multicore-friendly lightweight parallel framework" that supports a style of parallel programming where problems are recursively split into smaller fragments, solved in parallel and recombined, as described in this pseudocode from A Java Fork/Join Framework:

```
Result solve(Problem problem) {
  if (problem is small)
     directly solve problem
  else {
     split problem into independent parts
     fork new subtasks to solve each part
     join all subtasks
     compose result from subresults
  }
}
```

This ensures that the work can be accomplished in parallel and works well on multi-core systems:

> Big (32+) multicores will be coming out in volume during the lifetime of Java 7, and having this framework in place will allow people running them to get relatively simple

> speedups for computation-intensive tasks. Right now, forkjoin works best on machines like Sun Niagaras and Azuls, which are the only current processors of the form that will soon be common. It also works well on standard SMPs. In general, you don't get many speedup benefits with less than 4 processors though -- the main targets are in the dozens to hundreds of processors range.

Given a sufficiently-decomposed task size, the cost of executing the task may be exceeded by the cost of creating a thread. Accordingly, the fork/join framework uses thread pools which can be sized appropriately to the number of available cores to reduce the swapping overhead. To avoid straggling threads, the framework includes a work-stealing approach where a thread with no work can poach queued work from a slower-executing thread. The A Java Fork/Join Framework paper describes the mechanisms used by the fork/join framework and shows examples of performance in multicore systems and comparisons to other similiar kinds of parallel programming frameworks.

Describing the likely usage scenarios and adoption for the fork/join framework, Doug Lea said:

> Overall, I expect an adoption curve vaguely similar to other concurrency utilities. At first only a few people who really need them will use them, but eventually, it will be hard to find programs that don't somehow rely on them, often buried deep inside infrastructure components. For this reason, surface syntax support might not be all that important anyway -- the kinds of library/component developers most likely to want to incorporate them are more likely to put up with awkward usage.
>
> Ideally, there will be several layers of usages:
>
> > "Do this stuff in parallel" level, where a language or tools translates into parallel-ese, while also checking for safety/liveness. This is still partially in the realm of research though.
> >
> > Arranging for parallel operations on collections. -- map, reduce, apply, etc. Programmers who can think about manipulating collections using aggregate all-at-once operations can use these to speed up common kinds of processing. (This is the ListTasks, ArrayTasks, etc layer.)
> >
> > Hand-crafting an efficient forkjoin solution for a given problem. This is the layer I'm right now concentrating on, to be sure we can support the wide range of parallel algorithms possible using work-stealing frameworks. (Some of the currently odd-looking and under-explained methods like isQuiescent are designed for such advanced usages. Most programs just use "fork" and "join", but the others are there when you need them.)
> >
> > Extending the framework to create new kinds of ForkJoinTasks etc. For example, those that operate transactionally. Only a very small number of people (for example, perhaps the Fortress runtime library developers) will ever need to do this, but there need to be enough base extension hooks to pull this off.

Another candidate for Java SE 7 is the TransferQueue. TransferQueue extends the BlockingQueue interface by adding producer-side blocking where the producer transfers an object to the consumer when both are ready. Non-blocking and timeout-style transfers are also available.
This is suitable for some kinds of message-passing applications where a hand-off can be necessary, and "to create a simpler-to-use thread pool, in which sometimes tasks must be synchronously transfered"

The transfer queue and fork/join framework may not be the only concurrency features added to Java SE 7. According to Doug Lea, there is an ongoing wishlist of new and request-for-enhancement features, including scalable counters, a ConcurrentIdentityHashMap, "additions to the atomics package to better support expert-mode fine-tuning of memory model effects" and algorithmic improvements to java.util.Collections.

The jsr166y effort builds on the work done in Java SE 6 as jsr166x, which brought new

collections into Java
SE6: Deque, BlockingDeque, NavigableSet, NavigableMap andConcurrentNavigableMap (as well as the implementing
classes, ArrayDeque,LinkedBlockingDeque, ConcurrentSkipListSet, ConcurrentSkipListMap and enhancements toTreeMap).

Element 5:

the each of the fragments is processed till one or more of predefined conditions is met,

Adding a bit of drama to the slide-to-unlock debate, Micron has just received its own patent covering a "system and method for controlling user access to an electronic device." U.S. Patent No 8,352,745 issued in January 2013 but claims priority to an original application filed in February 2000 and lists Jim McKeeth as inventor.

Claims:

> 1. A system comprising:
>
> a touch screen upon which a user is to enter, by drawing, a geometric pattern in a specified direction to gain access to the system; and
>
> a processing circuit coupled to the touch screen to compare the user entered geometric pattern to a predefined geometric pattern stored in a memory.
>
> 6. The system of claim 1 wherein the user entered geometric pattern is a line.

Apple's slide-to-unlock patents claim priority only to the 2004-2005 timeframe and, although they may be separately patentable, Apple's patents will not serve to invalidate Micron's new patent.

Samsung has a 10-year cross-license deal with Micron. I have not seen reports of any similar deals with Apple, Google\Motorola, or RIM. It will be interesting to see whether Micron holds onto the patent or shifts it to an enforcement company as it did in 2010 in sending 4,000+ patents to John Desmarais for enforcement. Micron is a major manufacturer of computer memory with a market valuation of about $8 billion, perhaps they should just ask for Apple to endorse-over its recent billion dollar patent infringement judgment. One complicating factor is that I suspect Micron is a supplier for all of the potential defendants.

Element 6:

the predefined conditions include: the timer assigned to the each of the fragments is timed out

Abstract

XPath 2.0 is an expression language that allows the processing of values conforming to the data model defined in [XQuery 1.0 and XPath 2.0 Data Model (Second Edition)]. The data model provides a tree representation of XML documents as well as atomic values such as integers, strings, and booleans, and sequences that may contain both references to nodes in an XML document and atomic values. The result of an XPath expression may be a selection of nodes from the input documents, or an atomic value, or more generally, any sequence allowed by the data model. The name of the language derives from its most distinctive feature, the path expression, which provides a means of hierarchic addressing of the nodes in an XML tree. XPath 2.0 is a superset of[XPath 1.0], with the added capability to support a richer set of data

types, and to take advantage of the type information that becomes available when documents are validated using XML Schema. A backwards compatibility mode is provided to ensure that nearly all XPath 1.0 expressions continue to deliver the same result with XPath 2.0; exceptions to this policy are noted in [**I Backwards Compatibility with XPath 1.0**].

Status of this Document

This section describes the status of this document at the time of its publication. Other documents may supersede this document. A list of current W3C publications and the latest revision of this technical report can be found in the W3C technical reports indexat http://www.w3.org/TR/.

This is one document in a set of eight documents that are being progressed to Edited Recommendation together (XPath 2.0, XQuery 1.0, XQueryX 1.0, XSLT 2.0, Data Model (XDM), Functions and Operators, Formal Semantics, Serialization).

This document, published on 14 December 2010, is an Edited Recommendation of the W3C. It supersedes the previous W3C Recommendation of 23 January 2007. This second edition is not a new version of this specification; its purpose is to clarify a number of issues that have become apparent since the first edition was published. All of these clarifications (excepting trivial editorial fixes) have been published in a separate errata document, and published in a Proposed Edited Recommendation in April 2009. The changes are summarized in an appendix. On 3 January 2011, the original publication of this Recommendation was replaced by this version in which two HTML anchors that were omitted by the original publication have been restored; the W3C Team has retained a copy of the original publication. This document has been jointly developed by the W3C XSL Working Group and the W3C XML Query Working Group, each of which is part of the XML Activity.

This document has been reviewed by W3C Members, by software developers, and by other W3C groups and interested parties, and is endorsed by the Director as a W3C Recommendation. It is a stable document and may be used as reference material or cited from another document. W3C's role in making the Recommendation is to draw attention to the specification and to promote its widespread deployment. This enhances the functionality and interoperability of the Web.

This document incorporates changes made against the Recommendation of 23 January 2007 that resolve all errata known at the date of publication. Changes to this document since the first edition are detailed in the **J Changes since the First Edition**. This document supersedes the first edition.

This specification is designed to be referenced normatively from other specifications defining a host language for it; it is not intended to be implemented outside a host language. The implementability of this specification has been tested in the context of its normative inclusion in host languages defined by the XQuery 1.0 and XSLT 2.0 specifications; see the XQuery 1.0 implementation report and the XSLT 2.0 implementation report (member-only) for details.

Please report errors in and submit comments on this document using W3C's public Bugzilla system (instructions can be found at http://www.w3.org/XML/2005/04/qt-bugzilla). If access to that system is not feasible, you may send your comments to the W3C XSLT/XPath/XQuery public comments mailing list, public-qt-comments@w3.org. It will be very helpful if you include the string “[XPath]” in the subject line of your report, whether made in Bugzilla or in email. Each Bugzilla entry and email message should contain only one error report. Archives of the comments and responses are available at http://lists.w3.org/Archives/Public/public-qt-comments/.

This document was produced by groups operating under the 5 February 2004 W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the XML Query Working Group and also maintains a public list of any patent disclosures made in connection with the deliverables of the XSL Working Group; those pages also include instructions for disclosing a patent. An individual who has actual knowledge of a patent which the individual believes containsEssential Claim(s) must disclose the information in

accordance with section 6 of the W3C Patent Policy.

Element 7:

or an acknowledgement of at least one of the data blocks in the each of the fragments is received;

1 Introduction

Cluster-based storage systems are becoming an increasingly important target for both research and industry [*1*, *36*, *15*, *24*, *14*, *8*]. These storage systems consist of a networked set of smaller storage servers, with data spread across these servers to increase performance and reliability. Building these systems using commodity TCP/IP and Ethernet networks is attractive because of their low cost and ease-of-use, and because of the desire to share the bandwidth of a storage cluster over multiple compute clusters, visualization systems, and personal machines. Furthermore, non-IP storage networking lacks some of the mature capabilities and breadth of services available in IP networks. However, building storage systems on TCP/IP and Ethernet poses several challenges. In this paper, we analyze one important barrier to high-performance storage over TCP/IP: the *Incast* problem [*24*].[1]

Incast is a catastrophic TCP throughput collapse that occurs as the number of storage servers sending data to a client increases past the ability of an Ethernet switch to buffer packets. As we explore further in §2, the problem arises from a subtle interaction between limited Ethernet switch buffer sizes, the communication patterns common in cluster-based storage systems, and TCP's loss recovery mechanisms. Briefly put, data striping couples the behavior of multiple storage servers, so the system is limited by the request completion time of the *slowest* storage node [*7*]. Small Ethernet buffers are exhausted by a concurrent flood of traffic from many servers, which results in packet loss and one or more TCP timeouts. These timeouts impose a delay of hundreds of milliseconds–orders of magnitude greater than typical data fetch times–significantly degrading overall throughput.

This paper provides three contributions. First, we explore in detail the root causes of the *Incast* problem, characterizing its behavior under a variety of conditions (buffer space, varying numbers of servers, etc.). We find that *Incast* is a general barrier to increasing the number of source nodes in a cluster-based storage system. While increasing the amount of buffer space available can delay the onset of *Incast*, any particular switch configuration *will* have some maximum number of servers that can send simultaneously before throughput collapse occurs.

Second, we examine the effectiveness of existing TCP variants (e.g., Reno [*3*], NewReno [*13*], SACK [*22*], and limited transmit [*2*]) designed to improve the robustness of TCP's loss recovery. While we do find that the move from Reno to NewReno substantially improves performance, none of the additional improvements help. Fundamentally, when TCP loses *all* packets in its window or loses retransmissions, no clever loss recovery algorithms can help.

Third, we examine a set of techniques that are moderately effective in masking *Incast*, such as drastically reducing TCP's retransmission timeout timer. With some of these solutions, building a high-performance, scalable cluster storage system atop TCP/IP and Ethernet can be practical. Unfortunately, while these techniques can be effective, none of them is without drawbacks. Our final conclusion is that no existing solutions are entirely sufficient, and further research is clearly indicated to devise a principled solution for the *Incast* problem.

2 Background

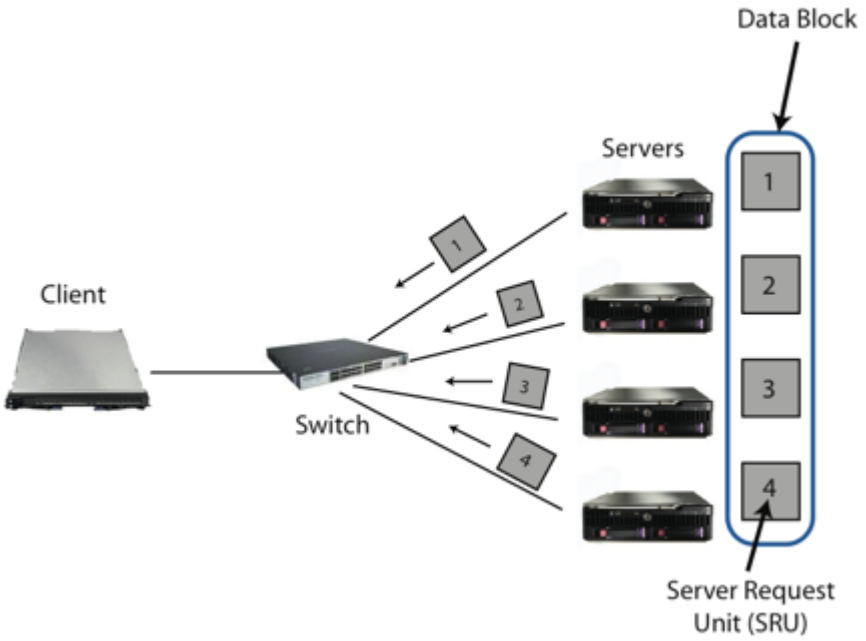
In cluster-based storage systems, data is stored across many storage servers to improve both

reliability and performance. Typically, their networks have high bandwidth (1-10 Gbps) and low latency (round trip times of tens to hundreds of microseconds) with clients separated from storage servers by one or more switches.

In this environment, data blocks are striped over a number of servers, such that each server stores a fragment of a data block, denoted as a *Server Request Unit (SRU)*, as shown in Figure 1. A client requesting a data block sends request packets to all of the storage servers containing data for that particular block; the client requests the next block only after it has received all the data for the current block. We refer to such reads as *synchronized reads*.

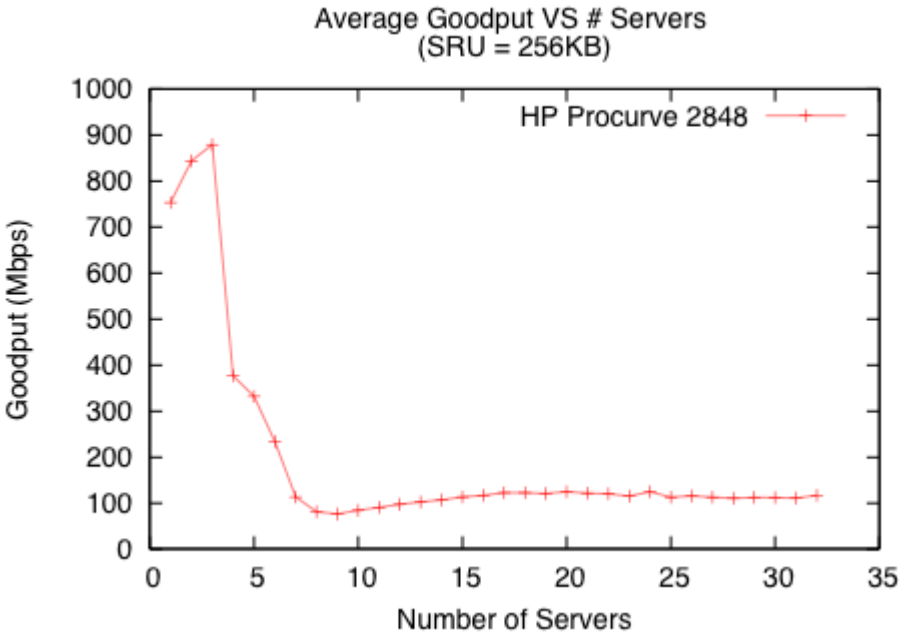
This simple environment abstracts away many details of real storage systems, such as multiple stripes per data block, multiple outstanding block requests from a client, and multiple clients on a single switch making requests across a shared subset of servers. However, this is the most basic representative setting in which *Incast* can occur and simplifies our analysis.

The need for a high performance environment that supports parallel operations such as synchronized reads is particularly important because of such recent projects as *Parallel NFS (pNFS)*. *pNFS* is a component of *NFSv4.1* that supports parallel data transfers and data striping across multiple file servers [*37*,*28*, *18*].



Most networks are provisioned so the client's link capacity to the switch is the throughput bottleneck of any parallel data transfer [*16*, *21*]. Unfortunately, when performing synchronized reads for data blocks across an increasing number of servers, a client may observe a TCP throughput drop of one or two orders of magnitude below its link capacity. Figure 2 illustrates this performance drop in a cluster-based storage network environment when a client requests data from just seven servers.

Most networks are provisioned so the client's link capacity to the switch is the throughput bottleneck of any parallel data transfer [*16*, *21*]. Unfortunately, when performing synchronized reads for data blocks across an increasing number of servers, a client may observe a TCP throughput drop of one or two orders of magnitude below its link capacity. Figure 2 illustrates this performance drop in a cluster-based storage network environment when a client requests data from just seven servers.



Early parallel network storage projects, such as the NASD project [15], observed TCP throughput collapse in cluster-based storage systems during synchronous data transfers. This was documented as part of a larger paper by Nagle et al. [24], who termed the problem *Incast* and attributed it to multiple senders overwhelming a fixed-size switch buffer. However, while Nagle demonstrated the problem and suggested that an alternative TCP implementation shows a modest improvement, a full analysis and measurement of the problem was not performed nor were possible solutions presented.

Incast has not been thoroughly studied. Current systems attempt to avoid TCP throughput collapse by limiting the number of servers involved in any block transfer, or by artificially limiting the rate at which they transfer data. These solutions, however, are typically specific to one configuration (e.g. a number of servers, data block sizes, link capacities, etc.), and thus are not robust to changes in the storage network environment.

3 Experimental Setup

In this section, we describe the simulation and real system environments where we measure the effects of *Incast* and the corresponding workloads that we use in both settings.

3.1 Simulation Environment

All of our simulations use *ns-2* [27], an event-driven network simulator that models networked applications at the packet granularity. Our default simulation configuration consists of one client and multiple servers all connected to the same switch as shown in Figure 1.

Table 1 shows the parameters and their corresponding default values that we vary in simulation. We choose a 256KB default *SRU* size to model a production storage system [9]. From our simulations, we obtain global and per-flow TCP statistics such as retransmission events, timeout events, TCP window sizes, and other TCP parameters to aid in our analysis of *Incast*.

Parameter	Default
Number of servers	—
SRU Size	256KB
Link Bandwidth	1 Gbps
Round Trip Time (RTT)	100µs

Per-port switch output buffer size	—
TCP Implementation: Reno, NewReno, SACK	NewReno
Limited Transmit	disabled
Duplicate-ACK threshold (da_{thresh})	3
Slow Start	enabled
RTO_{min}	200ms

Table 1: Simulation parameters with default settings.

Our test application performs synchronized reads over TCP in *ns-2* to model a typical striped file system data transfer operation. The client requests a data block from *n* servers by sending a request packet to each server for one *SRU* worth of data. When a client receives the entire data block of $n \cdot SRU$ total bytes, it immediately sends request packets for the next block. Each measurement runs for 20 seconds of simulated time, transferring enough data to accurately calculate throughput.

3.2 Cluster-based Storage Environment

Our experiments use a networked group of storage servers as configured in production storage systems. Our application performs the same synchronized reads protocol as in simulation and measures the achieved throughput. All systems have 1 Gbps links and a client-to-server Round Trip Time (RTT) of approximately 100µs. We evaluated three different storage clusters:

- **Procurve:** One or more HP Procurve 2848 Ethernet switches configured in a tree hierarchy connect a client to up to 64 servers, each running Linux 2.6.18 SMP.[2]
- **S50:** A Force10 S50 switch connects 48 Redhat4 Linux 2.6.9-22 machines on one switch (1 client, 47 servers).
- **E1200:** A Force10 E1200 switch with 672 ports with at least 1MB output buffer per port. This switch connects 88 Redhat4 Linux 2.6.9-22 machines (1 client, 87 servers).

For our workload and analysis, we keep the *SRU* size fixed while we scale the number of servers, implicitly increasing the data block size with the number of servers.[3]

4 Reproducing Incast

In this section, we first demonstrate *Incast* occurring in several real-world cluster-based storage environments. Using simulation, we then show that *Incast*is a generic problem and identify the causes of *Incast*. We find that the results obtained from our experimental setup validate our simulation results. Finally, we show that attempts to mitigate *Incast* by varying parameters such as switch buffer size and *SRU* size are incomplete solutions that either scale poorly or introduce system inefficiencies when interacting with a storage system.

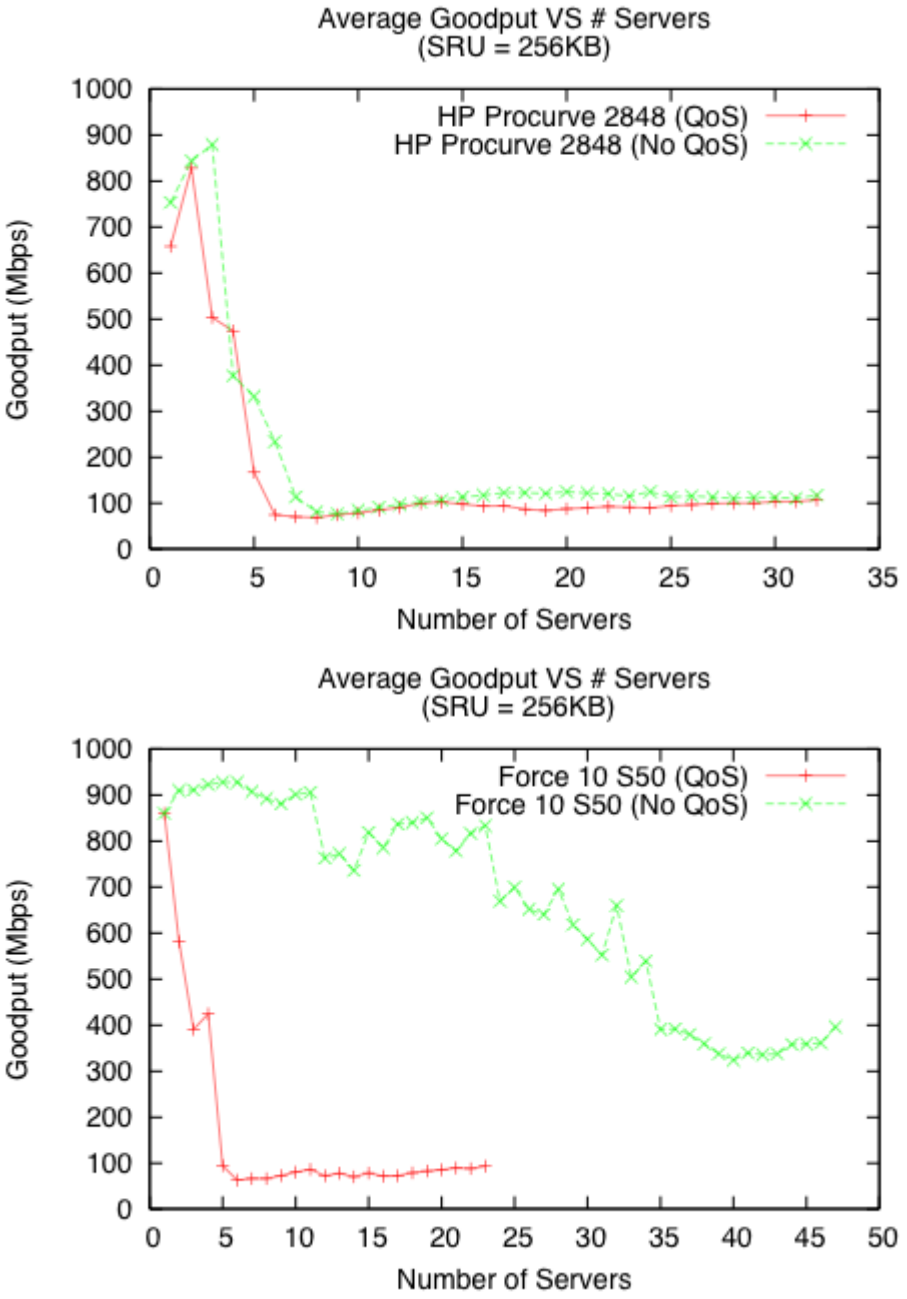


Figure 3: *Incast* **observed on different switch configurations with and without QoS support. Disabling QoS support has only a small effect for (a) the HP Procurve 2848, but significantly delays the onset of** *Incast* **for (b) the Force10 S50.**

4.1 *Incast* in real systems

To ensure that the throughput collapse shown in Figure 2 is not an isolated instance, we study *Incast* on the three storage clusters described in §3.2. Figure3 indicates that both the Procurve and S50 environments experience up to an order of magnitude drop in *goodput* (throughput as observed by the application). The E1200, however, did not exhibit any throughput drop for up to the 87 available servers, which we attribute to the large amount of buffer space available on the switch.

In our analysis, we use estimates of the output buffer sizes gathered from network administrators and switch specifications. Unfortunately, we are unable to determine the exact per-port buffer sizes on these switches.[4] This information is not available because most switches dynamically allocate each link's output buffer from a shared memory pool. Also, when QoS queues are enabled, the amount of memory allocated to the queues depends on vendor-specific implementations. However, our estimates for output buffer sizes are corroborated by simulation results.

Many switches provide QoS support to enable prioritization of different kinds of traffic. A

common implementation technique for providing QoS is to partition the output queue for each class of service. As a result, disabling QoS increases the effective size of the output queues, though the amount of this increase varies across different vendors and switches. As shown in Figure 3(a), disabling QoS on the Procurve environment does not significantly affect throughput – a collapse still occurs around 7 servers. In contrast, Figure 3(b) shows that disabling QoS on the Force10 S50 significantly delays the onset of*Incast*. These results suggest that the Force10 S50 allocates a relatively larger amount of buffer space and switch resources to QoS support in comparison to the Procurve 2848. Switch buffer sizes play an important role in mitigating *Incast*, as we evaluate in §4.3.

4.2 Validation and Analysis in Simulation

To determine how general a problem *Incast* is for cluster-based storage over TCP/IP/Ethernet, we also reproduce *Incast* in the *ns-2* network simulator. Figure 4 shows *Incast* in simulation with an order of magnitude collapse at around 8 servers and beyond. These results closely match those from the Procurve environment. The differences between the results, including the difference in behavior below 3 servers, have a few possible causes. First, simulated source nodes serve data as rapidly as the network can handle, while real systems often have other slight delays. We attribute the lower performance of the real system between 1-3 servers to these differences. Also, simulation does not model Ethernet switching behavior, which may introduce small timing and performance differences.

Despite these differences, real world experiments validate our simulation measurements, showing that the impact of *Incast* is nearly identical in both real world system measurements and simulation.

Element 8:

and a second module designed to start the timer assigned to the each of the fragments,

The Structure of OCaml Programs

Now we're going to take some time out to take a high-level look at some real OCaml programs. I want to teach you about local and global definitions, when to use `;;` vs. `;`, modules, nested functions, and references. For this we're going to look at a lot of OCaml concepts which won't yet make sense because we haven't seen them before. Don't worry about the details for the moment. Concentrate instead on the overall shape of the programs and the features which I'll point out.

Local "variables" (*really* local expressions)

Let's take the `average` function and add a local variable in C. (Compare it to the first definition we had above).

```
double average (double a, double b)
{
 double sum = a + b;
 return sum / 2;
}
```

Now let's do the same to our OCaml version:

```
# let average a b =
  let sum = a +. b in
  sum /. 2.0;;
val average : float -> float -> float =
```

The standard phrase `let name = expression in` is used to define a named local expression, and `name` can then be used later on in the function instead of `expression`, till a `;;` which ends the block of code. Notice that we don't indent after the `in`. Just think of `let ... in` as if it were a statement.

Now comparing C local variables and these named local expressions is a sleight of hand. In fact they are somewhat different. The C variable `sum` has a slot allocated for it on the stack. You can assign to `sum` later in the function if you want, or even take the address of `sum`. This is NOT true for the OCaml version. In the OCaml version, `sum` is just a shorthand name for the expression `a +. b`. There is no way to assign to `sum` or change its value in any way. (We'll see how you can do variables whose value changes in a minute).

Here's another example to make this clearer. The following two code snippets should return the same value (namely (a+b) + (a+b)²):

```
# let f a b =
  (a +. b) +. (a +. b) ** 2.;;
val f : float -> float -> float =
# let f a b =
  let x = a +. b in
  x +. x ** 2.;;
val f : float -> float -> float =
```

The second version might be faster (but most compilers ought to be able to perform this step of "common subexpression elimination" for you), and it is certainly easier to read. `x` in the second example is just shorthand for `a +. b`.

Element 9:

wherein the timer has a timeout value depending on a life of the each of the fragments

Open Caveats—Cisco ASR 1000 Series Aggregation Services Routers Release 3.9.2S

This section documents the unexpected behavior that may be seen in Cisco ASR 1000 Series Aggregation Services Routers Release 3.9.2S.

- CSCue34749

  Symptom: SIP crashes on RP switchover.

  Conditions: Issue is seen in scale setup.

  Workaround: SIP has to be reloaded.

- CSCue91054

  Symptom: ESP crashed when sending IPv6-fragmented traffic through DMVPN hub (MGRE tunnel).

  Conditions: This condition occurs when sending big IPv6 packets (need to do IPv6 fragmenation

after adding tunnel header) traffic through DMVPN hub. Large amout of IPv6 fragment traffic, for example, 5G on ESP20, which exceeds re-assembly performance number that is less than 2G.

Workaround: Change MTU to avoid IPv6 fragmentation.

- CSCuh18797

  Symptom: ESP crashed while removing policy-map from configuration. This issue is seen while removing the QoS configuration from standalone chassis and all the ports are down.

  Conditions: ESP crashed because of object-pending issue. This issue can only be reproduced when the QoS configuration is from NVRAM, and not when it is added on a live box. This may be related to ordering issue.

  On my 1RU router, just booted the QoS config, and remove the QoS policy will crash FP.

  QoS-1RU#conf t
  Enter configuration commands, one per line. End with CNTL/Z.
  QoS-1RU(config)#no policy-map
  PM-QOS-LAN-1 QoS-1RU(config)# *May 29 06:31:35 UTC: IOSXE_OIR-6-OFFLINECARD Card (fp) offline in slot F0do *May 29 06:31:46 UTC: CPPHA-3-FAULT SIP0: cpp_ha: CPP:0.0 desc:CPP Client process fail det:HA class:CLIENT_SW sev:FATAL id:1 cppstate:RUNNING res:UNKNOWN flags:0x0 cdmflags:0x0 *May 29 06:31:46 UTC: IOSXE-6-PLATFORM SIP0: cpp_ha: Shutting down CPP MDM while client(s) still connected *May 29 06:31:46 UTC: PMAN-3-PROCHOLDDOWN SIP0: pman.sh: The process fman_fp_image has been helddown (rc 134) *May 29 06:31:46 UTC: PMAN-0-PROCFAILCRIT SIP0: pvp.sh: A critical process fman_fp_image has failed (rc 134) *May 29 06:31:46 UTC: PMAN-3-PROCHOLDDOWN SIP0: pman.sh: The process cpp_ha_top_level_server has been helddown (rc 69) *May 29 06:31:48 UTC: CPPDRV-3-LOCKDOWN SIP0: cpp_cp: CPP10(0) CPP Driver LOCKDOWN due to fatal error. *May 29 06:31:50 UTC: IOSXE-6-PLATFORM SIP0: cpp_cdm: Shutting down CPP MDM while client(s) still connected *May 29 06:31:52 UTC: PMAN-5-EXITACTION SIP0: pvp.sh: Process manager is exiting: critical process fault, fman_fp_image, fp_0_0, rc=134May 29 06:33:03.763 R0/0: PMAN-5-EXITACTION Process manager is exiting: reload fru action requested
  Workaround: There is no workaround.

- CSCui38300

  Symptom: High latency is observed in customer network.

  Conditions: Under conditions such as forced test, it is possible to create scenarios where flow-lock contention is very high because of NAT gatekeeper failures.

  Workaround: There is no workaround.

- CSCuf74266

  Symptom: ASR-CUBE: Crash observed with DSMP.

  Conditions: Load scenario issue is observed.

  Workaround: There is no workaround.

- CSCug48145

  Symptom: ASR DTMF interworking failed after reinvite with the block configured.

  Conditions: DTMF configured with different preference results in issue.

  Workaround: There is no workaround.

- CSCug77212

  Symptom: ASR1K CUBE RP may crash with Segmentation fault(11), Process = CCSIP_SPI_CONTROL when SIP headers are manipulated using a SIP profile for 200 response

messages for KPML notify.

Conditions: Crash could be due to SIP profile configurations being wrongly applied to Notify response (this profile was meant for 200 OK Invite response).

Workaround: Do not configure SIP profiles to manipulate the headers for 200 responses.

- CSCui00417

  Symptom: Linear increase of CPU for CCSIP_SPI_CONTROL and AFW_application processes, and CPU does not stabilize.

  Conditions: Basic SIP calls (INVITE to BYE) run for 2 hours or more.

  Workaround: There is no workaround.

- CSCug19588

  Symptom: IKEv2 TPS performance degradation over time.

  Conditions: This occurs in the lab under extreme test conditions with traffic running during session bring-up.

  Workaround: Reduce traffic and session bring-up rate.

- CSCud78578

  Symptom: RP crashes after FP switchover.

  Conditions: FP(FP80) reload with QoS configurations and traffic flowing in the background.

  Workaround: There is no workaround.

- CSCuh20209

  Symptom: ucode crashes when running the clear ip nat translations command.

  Conditions: This condition occurs very rarely with stateful traffic.

  Workaround: Use clear ip nat translations vrf vrf_name command to clear VRF aware translations.

- CSCuh73422

  Symptom: ASR1k With MAP-T Configuration crashes.

  Conditions: When a Ping is initiated to public IPv4 address, the Cisco ASR 1000 Series Aggregation Services Routers crash with Core dump, and the packet is translated but the packet causes an ICMP error message to be generated. In case of ICMP error generation, the box could crash.

  Workaround: There is no workaround.

- CSCue14586

  Symptom: After reload, system may not be able to bring up all IPsec tunnels at high scale (1k) group members.

  Conditions: This condition occurs on the Cisco ASR 1000 Series Aggregation Services Routers with GETVPN, 1k group members.

  Workaround: Issue the clear crypto gdoi command to force re-registration and rebuild of tunnels.

- CSCue50484

Symptom: Crypto Tunnel Socket remains OPEN after shutting down the tunnel interface.

Conditions: This condition occurs on the Dual-DmVPN with ike-profile on the tunnel interface.

Workaround: There is no workaround.

- CSCuf96673

  Symptom: Memory leaks are seen with Smap-Dmap scale scenario for 4000 sessions.

  Conditions: Leaks are seen after stress testing in rekey, dpd, and clear commands.

  Workaround: There is no workaround to prevent memory leaks.

- CSCui21309

  Symptom: ASR RP Crash is observed.

  Conditions: This condition occurs on the Cisco ASR 1002-x Routers running on 3.8.2S. This ASR is configured as dVTI IPSec Server.

  Workaround: There is no workaround.

- CSCui26458

  Symptom: Call flow: **Verizon -- CUBE -- CUSP -- Genesys/IVR**, transfered with SIP Refer back to PSTN hair-pining the call on CUBE. When the call is put on hold to be transferred from IVR to PSTN, the CODEC negotiation fails, dropping the call with reason code 47 and hanging the UDP port used. All the subsequent calls that try to reuse the same UDP port for RTP stream are dropped with reason code 47 and provison RSP failure is logged on **show voip fpi stats** command.

  Conditions: Hair-pinned calls that receive multiple M-Lines on the SDP received from Verizon on the original SIP Invite.

  Workaround: There is no workaround. Reload of router is required to clear hung UDP ports.

- CSCue89779

  Symptom: A FlexVPN spoke configured with an inside VRF and front-door VRF may have problems with spoke-to-spoke tunnels if they are not the same. During tunnel negotiation, two virtual-access interfaces are created (while only one is needed), the one in excess may fail to cleanup correctly. As a result, the routes created by NHRP process may lead to loss of traffic, or traffic may continue to flow through the Hub.

  Conditions: This symptom occurs when the VRF used on the overlay (IVRF) and the VRF used on the transport (FVRF) are not the same.

  Workaround: There is no workaround.

- CSCuh56482

  Symptom: A client connects to the IOS headend when launched from the browser, but if the connection attempt is made directly from the client it fails.

  Conditions: This symptom occurs under the following conditions:

  Cert-only client authentication is configured on the IOS headend.

  Standalone client is used to connect.

  There is more than one client certificate on the host machine.

  Workaround: Launch the connection from the browser instead.

- CSCui07002

  Symptom: When two routers attempt to build an IKE session and use PKI for authentication, if the CRL has expired, the responding router crashes and reloads.

  Conditions: PKI chain-validation, CRL check, expired CRL

  Workaround: Disable CRL check.

- CSCui12338

  Symptom: PPPoX sessions are not coming up with scaled configurations above 24,000.

  Conditions: The Service Provider network should be NAT enabled to 200 VRF with MPLS running in the core.

  Workaround: There is no workaround.

- CSCuh92188

  Symptom: Active IOSD crashed because of BGP task, which eventually cause the entire RP to reboot. This issue is not related to BGP task.

  Conditions: This symptom is seen when active IOSD process crashes. The reason of IOSD crash could be anything.

  Workaround: There is no workaround.

- CSCuh46031

  Symptom: The Cisco ASR 1000 Series Aggregation Services Router sends a different Acct-Session-Id in the Access-Request and Accounting-Request for the same user.

  Conditions: Flex VPN IPSEC remote access is configured.

  Workaround: There is no workaround.

- CSCue48456

  Symptom: Call is disconnected through CUBE.

  Conditions: This symptom occurs on a video call where a mid-call re-INVITE occurs to modify the media stream.

  Workaround: There is no workaround.

- CSCuh90658

  Symptom: QFP crash.

  Conditions: This symptom occurs under the following conditions:

    Create normal GTPv1 session and primary PDP.

    Delete request with teardown false.

    Update QoS with different data TEID at both SGSN and GGSN when crash occurred.

  Workaround: There is no workaround.

- CSCuh95336

  Symptom: Second udpated PDP failed.

  Conditions: Update the second PDP normally with different data TEID.

Workaround: There is no workaround.

- CSCui22356

  Symptom: The Subpackage ISSU Upgrade is performed on ASR1002-X router after upgrading the standby RP (R0/1) with new RP subpackages. Then, Switchover is forced from the active IOS process to the standby IOS process. During the switchover, new active RP performs configuration Bulk-Sync with the standby RP. During this Bulk Sync operation, the configuration related to the interfaces is not synchronized with the standby RP due to Bulk Sync MCL failures. The following error message will be displayed when this error is present.

  Sample Error Message:

  <.....>
  Config Sync: Bulk-sync failure due to Servicing Incompatibility. Please check full list of mismatched commands via:
  show redundancy config-sync failures mcl
  Config Sync: Starting lines from MCL file:
  interface Tunnel150
  ! "interface"
  - tunnel source GigabitEthernet0/0/0.34
  <.....>
  Standby takes more time(~744 seconds) for reaching terminal State.
  Conditions: The symptom is observed after redundancy force-switchover step in ISSU upgrade procedure.

  Workaround: Peform a standby IOS reload using the following command:

  hw-module subslot R0/0 reload

Element 10:

and time parameters thereof

What kind of country variables are you thinking of? Do you want to calculate this correlation historically or only for the current day situation?

Be aware that whatever you do, if you simply try to derive some inference about the relationship between domestic country parameters (political systems, strength of economy, culture) and friendliness/hostility, you will run into problems by omitting a range of other factors that likely contribute to whether two (or more) countries are friendly or hostile towards each other. Most of all, as I mentioned in my other comment, if we think of the international system between states as a social network - and I believe that is a good approach - then it is probably right to assume that certain attributes of the nodes (i.e. states) in the network do contribute to the forming (or dissolution) of edges between them. Said differently, it appears likely that two countries' political systems, cultural affiliations, geographic proximity, etc., contribute to the edge-forming process. However, and this is where problems arise with models that only look at two states in isolation, network effects and interdependencies will likely also play a role. What this means is that, if we have three states A, B, and C, where A and B are friendly towards each other and A and C are hostile, this will have an effect on what B chooses to do. Maybe the domestic variables would tell B to associate more closely with C, however due to its alliance/good relations with A, it might decide to become hostile towards C instead.

These interdependency effects are not captured well with correlations or regressions that use tie formation as dependent variables and country variables on a country-pair basis as independent variables.

Conclusion, if this is only a small interest/personal research you're carrying out, you might get

away with a few simple correlations, however if this is done in a more rigorous research framework, I would caution against using just correlations or related approaches.

Edit: In order to provide something to actually help you move forward, here are some data sets you might use:

- The Correlates of War

Various historical data sets, including militarized interstate disputes which you might use to infer some kind of animosity between countries over time.

- UCDP/PRIO Armed Conflict Dataset

Similar to CoW, a data set about armed conflicts historically. Uses a different coding methodology to CoW.

- Varieties of Democracy Dataset

I am less familiar with this one, but it aims to code a historical dataset that captures different types of democracies over time.

- Global Database of Events, Language, and Tone (GDELT)

The last one is for only the case in which you feel adventurous. I haven't worked with it myself, but it's a whole different beast and requires significantly more coding, preprocessing, and managing on your part before you can use it for your purposes.

Element 11:

, wherein the timer is running until processing of the each of the fragments is terminated,

We cannot imagine Internet service without TCP/IP. All Internet services we have developed and used at NHNare based on a solid basis, TCP/IP. Understanding how data is transferred via the network will help you to improve performance through tuning, troubleshooting, or introduction to a new technology.

This article will describe the overall operation scheme of the network stack based on data flow and control flow in Linux OS and the hardware layer.

Key Characteristics of TCP/IP

How should I design a network protocol to transmit data quickly while keeping the data order without any data loss? TCP/IP has been designed with this consideration. The following are the key characteristics of TCP/IP required to understand the concept of the stack.

> **TCP and IP**
>
> Technically, since TCP and IP have different layer structures, it would be correct to describe them separately. However, here we will describe them as one.

1. CONNECTION-ORIENTED

First, a connection is made between two endpoints (local and remote) and then data is transferred. Here, the "TCP connection identifier" is a combination of addresses of the two endpoints, having type.

2. BIDIRECTIONAL BYTE STREAM

Bidirectional data communication is made by using byte stream.

3. IN-ORDER DELIVERY

A receiver receives data in the order of sending data from a sender. For that, the order of data is required. To mark the order, 32-bit integer data type is used.

4. RELIABILITY THROUGH ACK

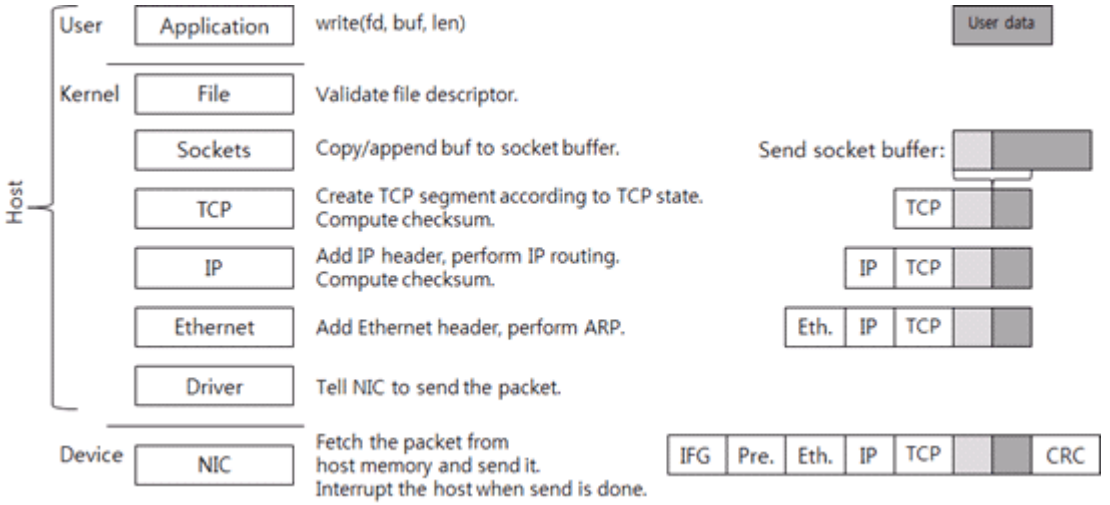
When a sender did not receive ACK (acknowledgement) from a receiver after sending data to the receiver, the sender TCP re-sends the data to the receiver. Therefore, the sender TCP buffers unacknowledged data from the receiver.

5. FLOW CONTROL

A sender sends as much data as a receiver can afford. A receiver sends the maximum number of bytes that it can receive (*unused buffer size*, *receive window*) to the sender. The sender sends as much data as the size of bytes that the receiver's *receive window* allows.

6. CONGESTION CONTROL

The congestion window is used separately from the *receive window* to prevent network congestion by limiting the volume of data flowing in the network. Like the receive window, the sender sends as much data as the size of bytes that the receiver's *congestion window* allows by using a variety of algorithms such as TCP Vegas, Westwood, BIC, and CUBIC. Different from flow control, congestion control is implemented by the sender only.



Element 12:

the timer is then reassigned to a next one of the fragments before the next one of the fragments is processed.

Use cases

- **UC_SV10** HLTSV receives L1-Result from RoIB

    - Or, the sw RoIB component running inside the HLTSV receives data from n S-links and assembles a L1-Result fragment

- **UC_SV20** HLTSV delivers a L1-Result to a DCM running on one of the available HLT nodes
- **UC_SV30** HLTSV receives a clear message from DCM when the latter has either fully collected an event or has rejected it
- **UC_SV31** HLTSV re-assigns an event to another DCM in case of s/w or h/w problems on the initial node
- **UC_SV40** HLTSV groups and broadcasts `ROBs-clear` messages to ROS PCs.
- **UC_SV50** HLTSV receives a message from DCM when a processing slot (a cpu core) is available
    - N.B.: during HLT processing after full data collection, the fragments can be deleted from ROBs, but a cpu core is still busy in EF processing

User requirements

- **UR_SV01: HLT input rate**
  The HLT system *shall* be able to sustain an input rate of 100kHz
- **UR_SV02: Single HLTSV**
  A single HLTSV instance *should* be able to receive L1-Results at 100kHz and distribute them to the HLT processing nodes
    - If the requirement cannot be satisfied, the HLT farm should be divided in sub-farms, each managed by its own HLTSV
- **UR_SV03: HLT farm balancing**
  The HLTSV *shall* take care of the HLT farm balancing
- **UR_SV04: Software based RoIB**
  It *should* be possible to merge the HLTSV with a software based RoIB. More...
- **UR_SV10 Condition data injection**
  The HLTSV *should* be an injection point for condition data changes. More...
- **UR_SV20: Event ownership**
  The HLTSV is in charge of an event (L1ID) until it is rejected by the HLT or fully collected by the DCM
- **UR_SV21: Fault tolerance**
  The HLTSV *shall* be able to identify h/w or s/w problems in an HLT node and re-assign the event allocated to the given node to another machine in the farm. More...
- **UR_SV22: Event reassignment**
  Reassigned events *shall* be properly labeled to prevent data duplication. More...
- **UR_SV40: ROB occupancy load**
  The HLTSV *shall* inform the ROSs when the fragments of a given event can be deleted.
    - I.e. when an event has been rejected or fully collected by an HLT node.
- **UR_SV41: ROS load**
  The HLTSV *should* reduce as much as possible the load on the ROS PC.
    - In order to reduce the ROS PC load, the HLTSV *should* group the `ROB-clear` messages. The grouping *shall* not contribute significantly to the ROB occupancy.
- **UR_SV50: Occupancy and deadtime**
  The HLTSV *shall* provide information about its buffers occupancy and dead-time

* * * * *

End of Report

Please rank my report. The following link leads to our profile at ClaimCharts.info, where you could rank our report.
Rank Here

Evaluation Code: MTZANUAyNkAyMQ==

We are available to answer follow up questions, provide clarifications and search for additional materials upon request. For public contact - please refer to our profile at ClaimCharts.info. For private contact - please call/email us.

Our hourly rate is: $

Non-Disclosure Agreement:

We understand that for the purpose of assisting you in collecting data, you may disclose to us certain confidential information that you want to keep confidential; therefore, for good and valuable consideration, the receipt and sufficiency of which is hereby acknowledged, we agree as follows:

1. For a period of twelve (36) months from the date we receive your confidential information ("Effective Date"), we will not use or disclose the information, except: (a) in the event that such information (i) comes into the public domain not through us; (ii) is already known to us at the time of disclosure; (iii) is lawfully otherwise learnt or independently developed by us; or (iv) is required to be disclosed by law.

2. Since a breach of our obligations hereunder may result in irreparable harm to you, we agree that upon such breach or threat thereof, you shall be entitled to specific performance and other injunctive relief without posting a bond, and in the event of disclosures required by court order, we shall promptly notify you of the proceeding.

3. This agreement supersedes all prior discussions and writings and constitutes the entire agreement between you and us with respect to the subject matter hereof. This Agreement shall be binding upon the successors and assigns of the respective parties, and may not be modified except by an instrument executed by both parties.

Alon Anthony